

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Интеллектуальные методы анализа данных»
Тема: Реализация метода классификации данных с использованием
природных временных рядов

Студент гр. 3384

Горский К.Д.

Преподаватель

Мандрикова Б.С.

Санкт-Петербург

2026

Цель работы.

Построение классификатора данных геофизического монитора на примере наблюдений нейтронных мониторов.

Задание.

Вариант 5. Тикси (ТХВУ), 10-19 августа 2015 г.

1. Подготовить данные. Скачать данные из источника [<https://www.nmdb.eu/nest/>] за периоды и станции, указанные по вариантам в таблице. Использовать данные с одномоментным временным разрешением. При наличии пропусков в данных заполнить их медианным значением (функция `median` в MatLab).
2. Сформировать векторы-истории. Временной ряд разбить на последовательные блоки (векторы) длиной m , каждый из которых должен содержать m прошлых значений. Последний вектор в ряде будет «тестовой» точкой для поиска соседей. Величину m выбрать равной 180 отсчетов.
3. Найти ближайших соседей. Для «тестового» вектора вычислить расстояние (метрику) до всех остальных векторов-историй (использовать 3 различные метрики). Число соседей (k) определить равным 2-4.
4. Использовать соседей для прогноза. На основе найденных ближайших соседей построить предсказания для будущих значений ряда. При нахождении нескольких схожих отрезков нужно взять простое среднее следующих за ними значений.
5. Сделать соответствующие выводы, оформить отчет по работе.

Выполнение работы.

Работа выполнялась на языке Python с модулем `pandas` для работы с массивами данных и `matplotlib` для визуализации данных. Исходный код см. в приложении А.

Сначала необходимо загрузить данные и подготовить их. Это выполняется также, как и в предыдущей лабораторной работе. Пропуски и выбросы (точки вне 3-сигма диапазона) заменяются на медианное значение.

Последний вектор является «тестовым». Для получения интересных результатов можно попробовать предсказать его всего (последние 180 точек данных).

Для краткости «векторы-истории» будут называться «окнами». Можно взять 180 окон, начиная с предпоследнего, и для каждого окна пытаться предсказывать следующее число. Таким образом, будут предсказаны 180 последних чисел. Для каждого из этих окон необходимо:

- Посчитать расстояние до всех остальных окон;
- Выбрать k ближайших окон;
- Для каждого из этих k окон взять следующее за ним число;
- Найти среднее этих k чисел.

Полученное среднее будет предсказанием числа, следующего за окном. Важно заметить, что при выборе k ближайших окон не надо рассматривать само интересующее нас в данный момент окно. Расстояние между окном и им самим всегда будет 0. Предсказываемое значение не должно участвовать в предсказании самого себя.

Для оценки качества прогноза последних 180 значений ряда использовались три меры расстояния (евклидово, манхэттенское, косинусное) и число ближайших соседей k от 2 до 4. Для каждой конфигурации рассчитаны средняя абсолютная ошибка (MAE), среднеквадратичная ошибка (RMSE) и коэффициент корреляции Пирсона между прогнозом и реальными значениями.

Таблица 1 — Результаты прогноза последних 180 значений ряда

Метрика	k	MAE	RMSE	Корреляция
Euclidean	2	1.694	2.121	-0.110
Euclidean	3	1.533	1.946	-0.051
Euclidean	4	1.481	1.849	-0.026
Manhattan	2	1.676	2.136	-0.037
Manhattan	3	1.550	1.988	-0.062
Manhattan	4	1.513	1.930	-0.115
Cosine	2	1.638	2.117	+0.031
Cosine	3	1.511	1.926	+0.071
Cosine	4	1.440	1.829	+0.104

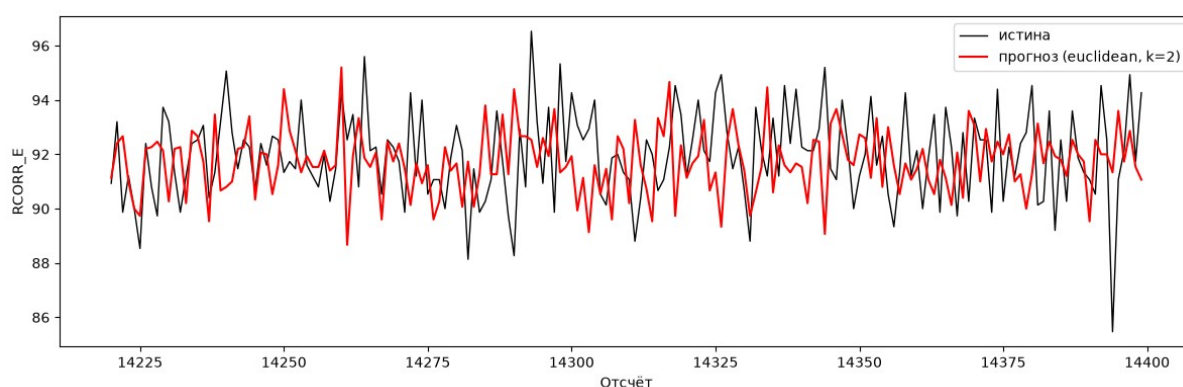


Рисунок 1 — Прогноз последних 180 значений (евклидова метрика, k=2)

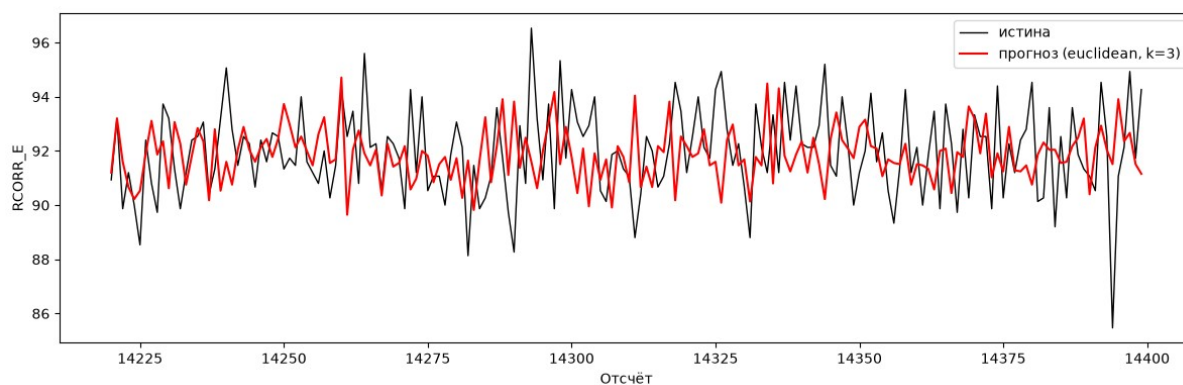


Рисунок 2 — Прогноз последних 180 значений (евклидова метрика, k=3)

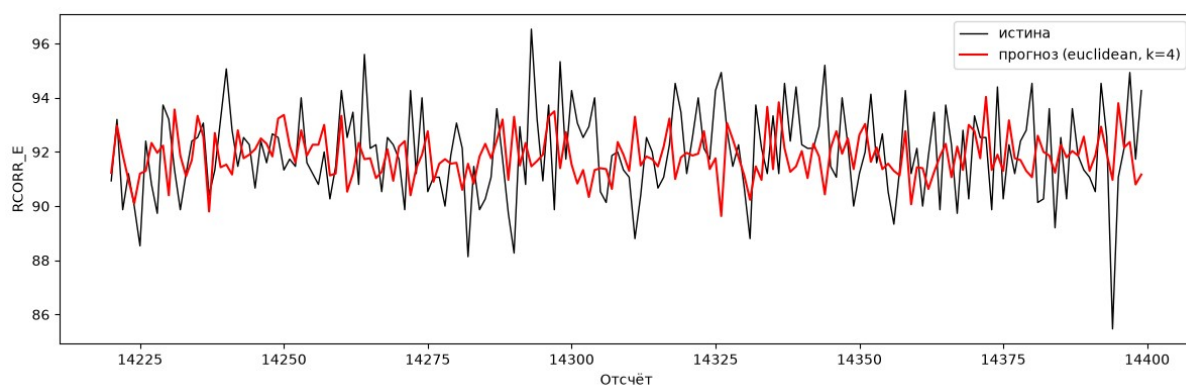


Рисунок 3 — Прогноз последних 180 значений (евклидова метрика, $k=4$)

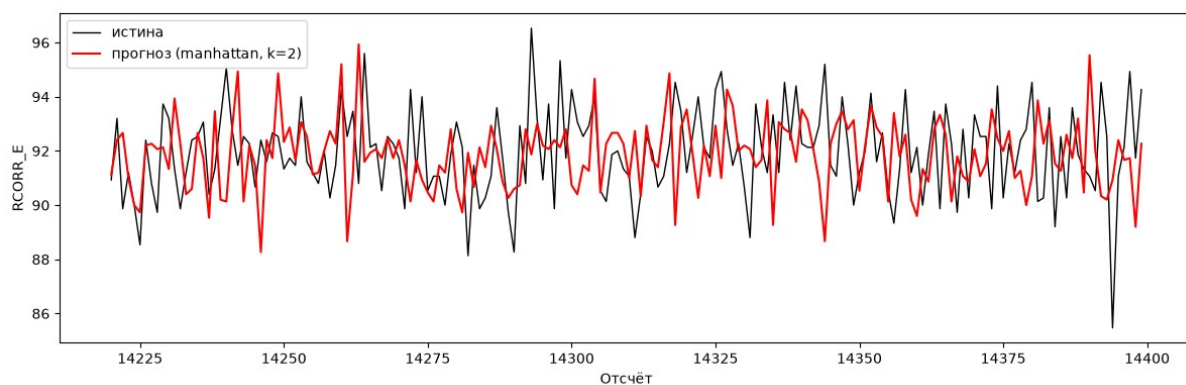


Рисунок 4 — Прогноз последних 180 значений (манхэттенская метрика, $k=2$)

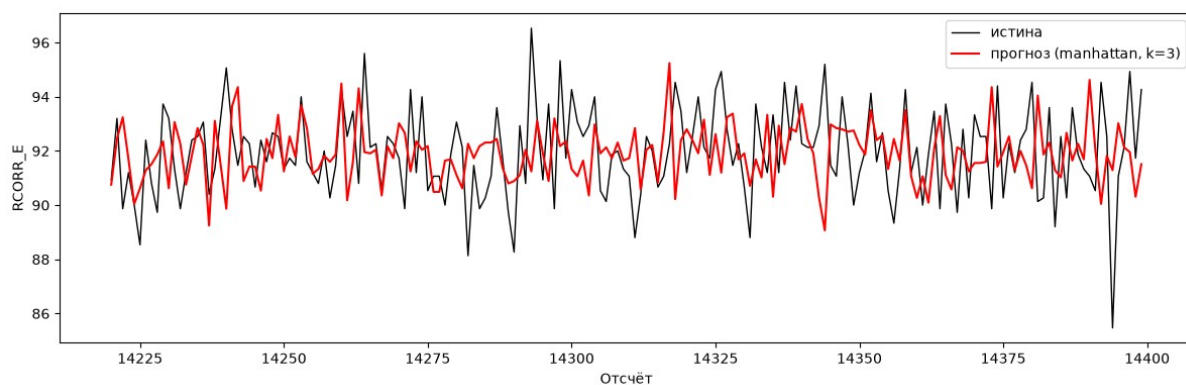


Рисунок 5 — Прогноз последних 180 значений (манхэттенская метрика, $k=3$)

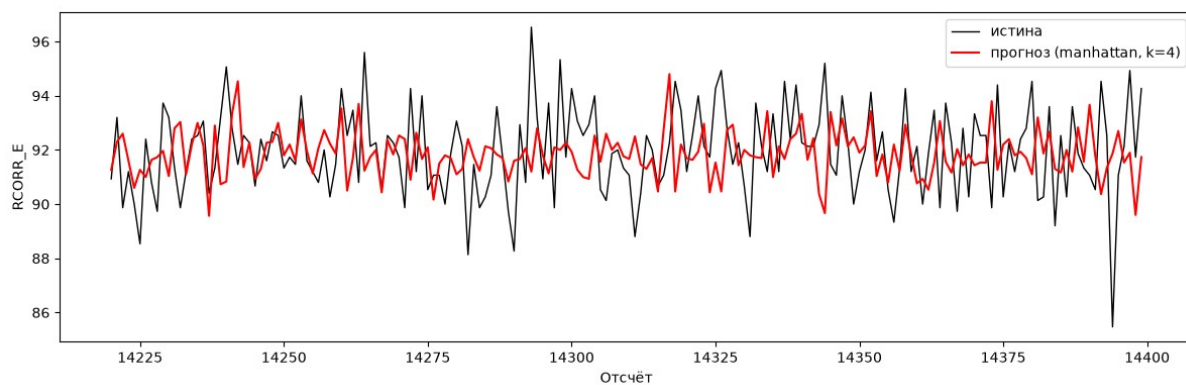


Рисунок 6 — Прогноз последних 180 значений (манхэттенская метрика, $k=4$)

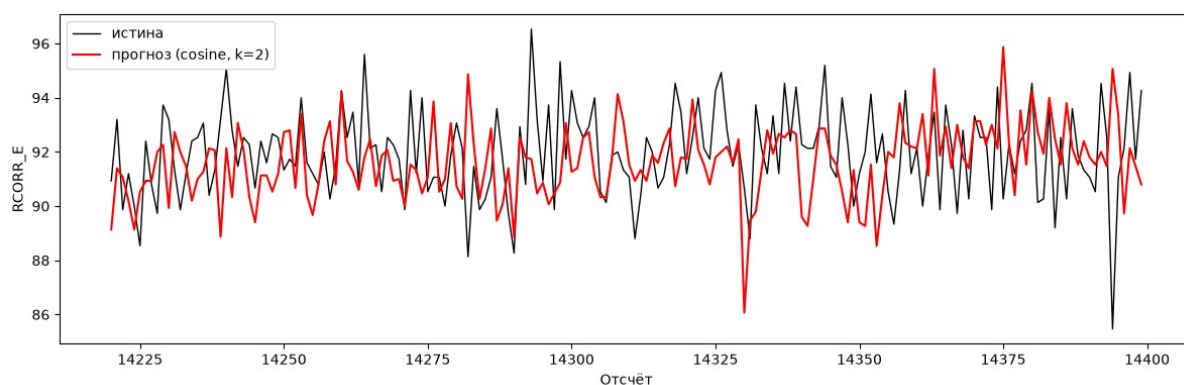


Рисунок 7 — Прогноз последних 180 значений (косинусная метрика, $k=2$)

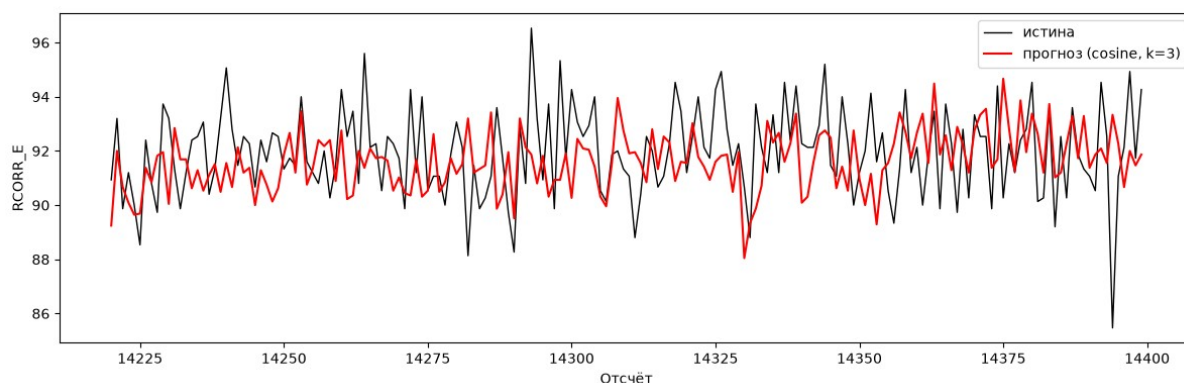


Рисунок 8 — Прогноз последних 180 значений (косинусная метрика, $k=3$)

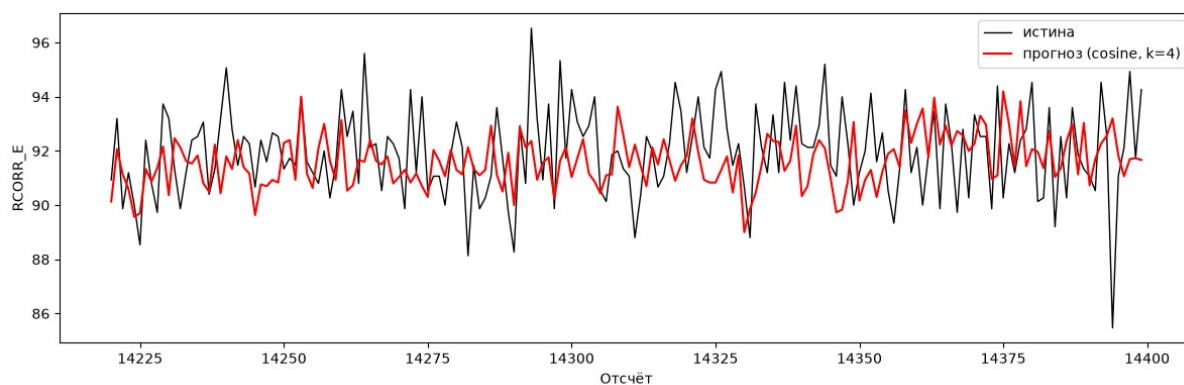


Рисунок 9 — Прогноз последних 180 значений (косинусная метрика, $k=4$)

Результаты прогнозов приведены в таблице 1, полученные графики приведены на рис. 1–9.

Из таблицы 1 видно, что с ростом числа соседей k ошибка прогноза уменьшается: MAE снижается примерно с 1.7 при $k=2$ до 1.44...1.51 при $k=4$. Различия между метриками незначительны и лежат в пределах 0.08 по MAE, то есть выбор меры близости почти не влияет на результат. Корреляция прогноза с

реальными значениями близка к нулю у всех конфигураций. Это значит, что метод не смог извлечь из данных никаких повторяющихся 180-минутных структур, а усреднение по ближайшим соседям сводится к предсказанию среднего уровня и не улавливает отдельные выбросы.

На рисунках 1–9 видно, что прогнозируемая кривая повторяет средний уровень ряда и практически не отслеживает отдельные колебания.

Выводы.

Построен классификатор данных геофизического монитора на примере наблюдений нейтронных мониторов. Реализован прогноз значений ряда методом k ближайших соседей.

С ростом k ошибка уменьшается — усреднение по большему числу соседей сглаживает влияние шума. Выбор метрики (евклидово, манхэттенское, косинусное расстояние) слабо влияет на результат. Корреляция прогноза с реальностью близка к нулю.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tqdm

M = 180
K_VALUES = [2, 3, 4]

def euclidean(a, b):
    d = a - b
    return np.sqrt((d * d).sum())

def manhattan(a, b):
    return np.abs(a - b).sum()

def cosine(a, b):
    denom = np.linalg.norm(a) * np.linalg.norm(b)
    return 1.0 - (a @ b.T) / denom

DIST_FUNCS = [
    ("euclidean", euclidean),
    ("manhattan", manhattan),
    ("cosine", cosine)
]

df = pd.read_csv("data.csv", skiprows=25, sep=";", names=["datetime",
"RCORR_E"])
df["datetime"] = pd.to_datetime(df["datetime"])
df["RCORR_E"] = pd.to_numeric(df["RCORR_E"])

mean = df["RCORR_E"].mean()
std = df["RCORR_E"].std()
range_min = float(mean - 3 * std)
range_max = float(mean + 3 * std)
mask_outliers = (df["RCORR_E"] < range_min) | (df["RCORR_E"] > range_max)

df_clean = df["RCORR_E"].drop(df[mask_outliers].index)
median = df_clean.median()

df.loc[mask_outliers, "RCORR_E"] = median
full = pd.date_range(df["datetime"].min(), df["datetime"].max(), freq="1min")
df = df.set_index("datetime").reindex(full)
df["RCORR_E"] = df["RCORR_E"].fillna(median)

series = df["RCORR_E"].to_numpy(dtype=float)
N = len(series)

history_vecs = [series[i:i + M] for i in range(N - M + 1)]

predict_targets = range(N - M - M, N - M)

true_vals = series[N - M:]
preds = {}

for norm_name, norm in DIST_FUNCS:
    print(f"norm={norm_name}")
    preds[norm_name] = {k: [] for k in K_VALUES}
    for j in tqdm.tqdm(predict_targets):
```

```

query = history_vecs[j]
prev_history_vecs = history_vecs[:j]
dists = np.array([norm(query, w) for w in prev_history_vecs])
order = np.argsort(dists)
for k in K_VALUES:
    nb = order[:k]
    preds[norm_name][k].append(series[nb + M].mean())

results = {}
for norm_name, _ in DIST_FUNCS:
    for k in K_VALUES:
        p = np.array(preds[norm_name][k])
        err = p - true_vals
        mae = float(np.mean(np.abs(err)))
        rmse = float(np.sqrt(np.mean(err ** 2)))
        corr = float(np.corrcoef(p, true_vals)[0, 1])
        results[(norm_name, k)] = (mae, rmse, corr, p)

    print(f"{norm_name} k={k}: MAE={mae:.3f}, RMSE={rmse:.3f}, corr={corr:
+.3f}")

fig, ax = plt.subplots(figsize=(12, 4))
t = np.arange(N - M, N)
ax.plot(t, true_vals, label="истина", lw=1.0, c="black")
ax.plot(t, p, label=f"прогноз ({norm_name}, k={k})", c="red")
ax.set_xlabel("Отсчёт");
ax.set_ylabel("RCORR_E")
ax.legend()
fig.tight_layout()
fig.savefig(f"img/predict_{norm_name}_k{k}.png")

```